

Unidad 9

Arquitectura U-net para datos espaciales. Concepto de Segmentación Semántica: Diferencia entre clasificación de imágenes, detección de objetos y segmentación píxel a píxel. Encoder (Camino de Contracción): Captura del contexto mediante convoluciones y pooling (reducción de resolución). Bottleneck (Cuello de Botella): Representación latente de máxima compresión. Decoder (Camino de Expansión): Recuperación de la resolución espacial mediante *Up-sampling* o Convoluciones Transpuestas. Skip Connections (Conexiones de Salto): La clave de la U-Net. Funciones de Pérdida Específicas: para evaluar segmentación. Aplicaciones espaciales.

Transfer learning

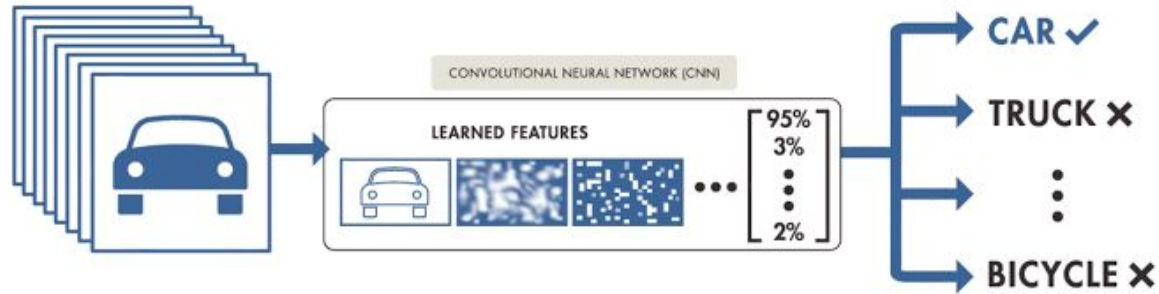
El **transfer learning** (o aprendizaje por transferencia) es la **transferencia del conocimiento que una red neuronal ha adquirido tras ser entrenada en un problema o dominio específico (usualmente con una inmensa cantidad de datos) para aplicarlo a una tarea nueva y diferente (donde los datos suelen ser más escasos)**. En la práctica, esto significa tomar un modelo que ya ha sido entrenado por otros expertos en un gran conjunto de datos y utilizarlo como punto de partida para tu propio problema.

¿Cómo funciona y cuál es su intuición? La idea central radica en que, si un modelo se entrena en un conjunto de datos lo suficientemente masivo y diverso (como ImageNet, que cuenta con más de un millón de imágenes), **el modelo aprende a extraer características genéricas que sirven como una representación excelente del mundo visual.**

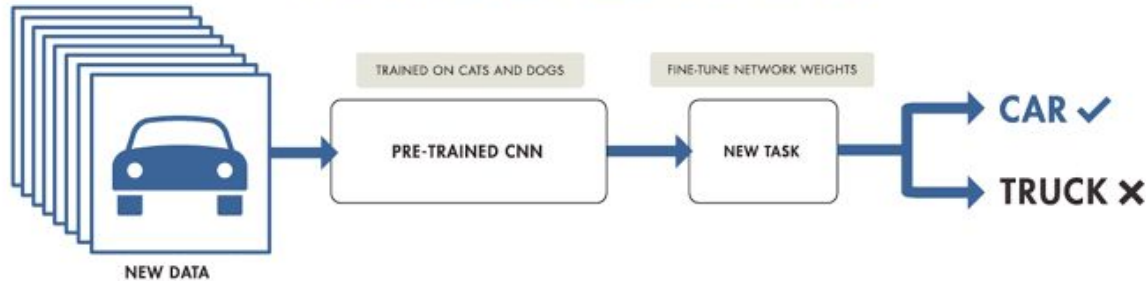
Como hemos hablado en definiciones anteriores sobre CNNs, las primeras capas de la red aprenden a detectar patrones visuales fundamentales (líneas, gradientes, bordes), mientras que las capas más profundas identifican formas complejas (ojos, ruedas, texturas). En lugar de inicializar los parámetros de una red desde cero con valores aleatorios y enseñarle a "ver" de nuevo, el *transfer learning* reutiliza estos mapas de características (feature maps) y pesos que ya han sido optimizados.

Transfer learning

TRAINING FROM SCRATCH



TRANSFER LEARNING



Fine-tuning (o ajuste fino) es una técnica que consiste en adaptar y actualizar los parámetros de un modelo preentrenado para que aprenda a resolver una tarea nueva y diferente.

En lugar de entrenar una red neuronal desde cero inicializando sus pesos al azar, el *fine-tuning* aprovecha los pesos que ya fueron optimizados previamente en un conjunto de datos masivo. Formalmente, el proceso implica "descongelar" (*unfreeze*) algunas de las capas superiores del modelo base (las encargadas de extraer características) y **entrenarlas de manera conjunta con las nuevas capas clasificadoras** que se añaden específicamente para la nueva tarea.

Se denomina "ajuste fino" porque su objetivo es **ajustar o modificar ligeramente las representaciones más abstractas del modelo original para hacerlas más relevantes y específicas al nuevo problema**, sin desechar las capacidades visuales o de lenguaje que ya había adquirido.

- **Ahorro drástico de datos, tiempo y costo computacional:** Es posiblemente el método más importante en el aprendizaje profundo moderno para lograr modelos altamente precisos de forma rápida, sorteando la necesidad de tener millones de imágenes etiquetadas y procesadores costosos.
- **Convergencia más rápida:** Al iniciar el entrenamiento con pesos que ya están cerca de un estado óptimo (en lugar de ser números aleatorios), el algoritmo de optimización llega a una buena solución en una fracción del tiempo.
- **Mejor generalización y prevención del sobreajuste:** Como el modelo base ya ha "visto" incontables escenarios y variaciones en su entrenamiento original, las características que se transfieren son muy robustas. Esto ayuda a que tu nuevo modelo sea menos propenso a memorizar tu pequeño conjunto de datos local (*overfitting*) y se desempeñe mejor ante situaciones no vistas en el mundo real.

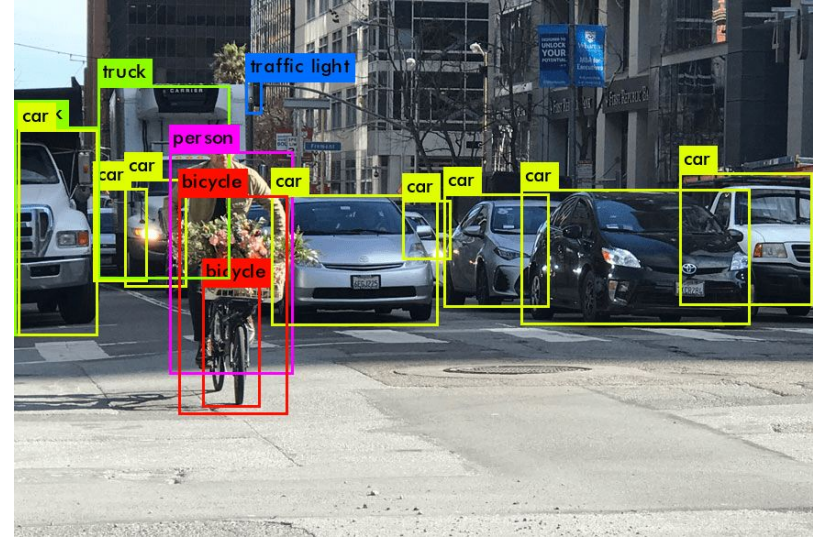
- **Red preentrenada como clasificador:** Se utiliza el modelo preentrenado exactamente como está, sin realizar ningún entrenamiento adicional ni modificar sus capas. Simplemente se introducen los datos nuevos y se utilizan las predicciones directas (útil cuando ambos problemas son extremadamente similares).
- **Red preentrenada como extractor de características:** Dado que la capa final (la capa densa o "cabeza") está altamente especializada en predecir las clases originales, **esta capa se elimina y se reemplaza por una nueva con pesos aleatorios que tenga la cantidad de salidas correctas para tu problema.** En este enfoque, **se "congelan" (*freeze*) todas las capas convolucionales previas** para impedir que modifiquen sus pesos, y el proceso de entrenamiento se enfoca únicamente en enseñar a tu nueva capa final a combinar esas características transferidas.

- **Fine-tuning (Ajuste fino):** Va un paso más allá del enfoque anterior. Además de entrenar tu nueva capa clasificadora, **se descongelan algunas de las últimas capas convolucionales del modelo base** (o incluso el modelo completo) y se entrenan de manera conjunta. Esto permite que el modelo adapte sutilmente las representaciones más abstractas para hacerlas más relevantes a los detalles específicos de tu nuevo conjunto de datos. Al hacer *fine-tuning*, es una práctica estándar **utilizar una tasa de aprendizaje (*learning rate*) mucho más baja** para evitar destruir rápidamente el valioso conocimiento previo almacenado en los pesos originales.

*Object
detection*

Detección de objetos

La **detección de objetos** (object detection) es una tarea fundamental de la visión artificial que va un paso más allá de la clasificación de imágenes tradicional. Mientras que la clasificación se centra en identificar cuál es el objeto principal en una imagen (por ejemplo, decir "gato"), la detección de objetos se encarga de localizar e identificar simultáneamente múltiples objetos dentro de una misma imagen, encerrando a cada uno en una **caja delimitadora** (bounding box) y asignándole su clase correspondiente.



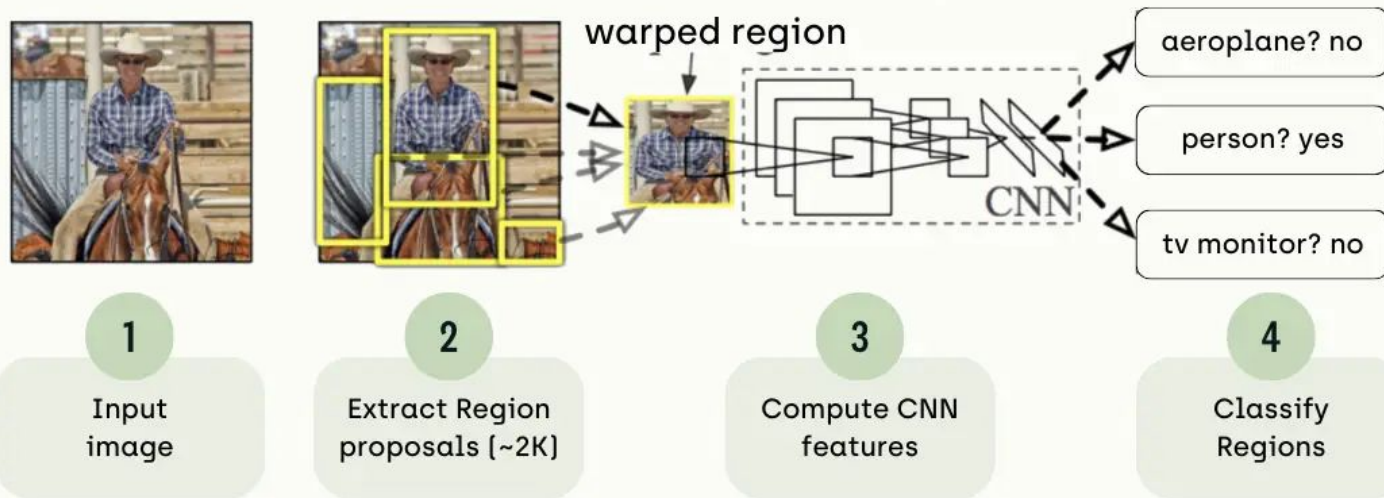
Arquitecturas para detección de objetos

Las redes diseñadas para la detección de objetos se dividen principalmente en dos enfoques:

1. Detectores de dos etapas (Familia R-CNN) Son modelos de detección divididos en fases secuenciales. Incluyen arquitecturas como **R-CNN**, **Fast R-CNN** y **Faster R-CNN**. En la primera etapa, una red o algoritmo propone las regiones candidatas escasas y, en la segunda etapa, estas regiones pasan por un clasificador y un regresor de cajas.

- **Ventaja:** Ofrecen una precisión sumamente alta.
- **Desventaja:** Son computacionalmente muy pesados y lentos. Por ejemplo, R-CNN evaluaba individualmente miles de recortes de la imagen a través de la CNN, siendo inviable para tiempo real. Aunque Faster R-CNN solucionó muchos cuellos de botella implementando una red convolucional exclusiva para proponer las regiones (RPN), sigue siendo más lento que las alternativas de una etapa.

R-CNN: Regions with CNN features



R-CNN (Region-based Convolutional Neural Network) es una arquitectura pionera diseñada para la tarea de detección y localización de objetos en imágenes, que combina algoritmos de propuesta de regiones con el poder de extracción de características de las redes neuronales convolucionales.

Módulos principales:

1. **Propuesta de regiones (Region Proposals):** Utiliza un algoritmo llamado búsqueda selectiva (*selective search*) para analizar la imagen original y generar alrededor de 2,000 regiones candidatas o "Regiones de Interés" (Rols) que tienen una alta probabilidad de contener un bloque u objeto. Posteriormente, estas regiones se deforman o redimensionan (*warp*) a un tamaño fijo.
2. **Extracción de características:** Cada una de estas 2,000 regiones redimensionadas se introduce de forma individual en una Red Neuronal Convolucional (CNN) preentrenada, la cual se encarga de extraer las características visuales de cada región candidata.
3. **Módulo de clasificación:** Las características extraídas por la CNN se pasan a un clasificador tradicional de aprendizaje automático llamado Máquina de Vectores de Soporte (SVM). Este algoritmo se encarga de predecir a qué categoría pertenece el objeto detectado.
4. **Módulo de localización (Regresión):** Paralelamente, se utiliza un algoritmo de regresión lineal para predecir y refinar las cuatro coordenadas exactas ($x, y, ancho, alto$) que conforman la caja delimitadora (*bounding box*) que encierra al objeto.

Fast R-CNN

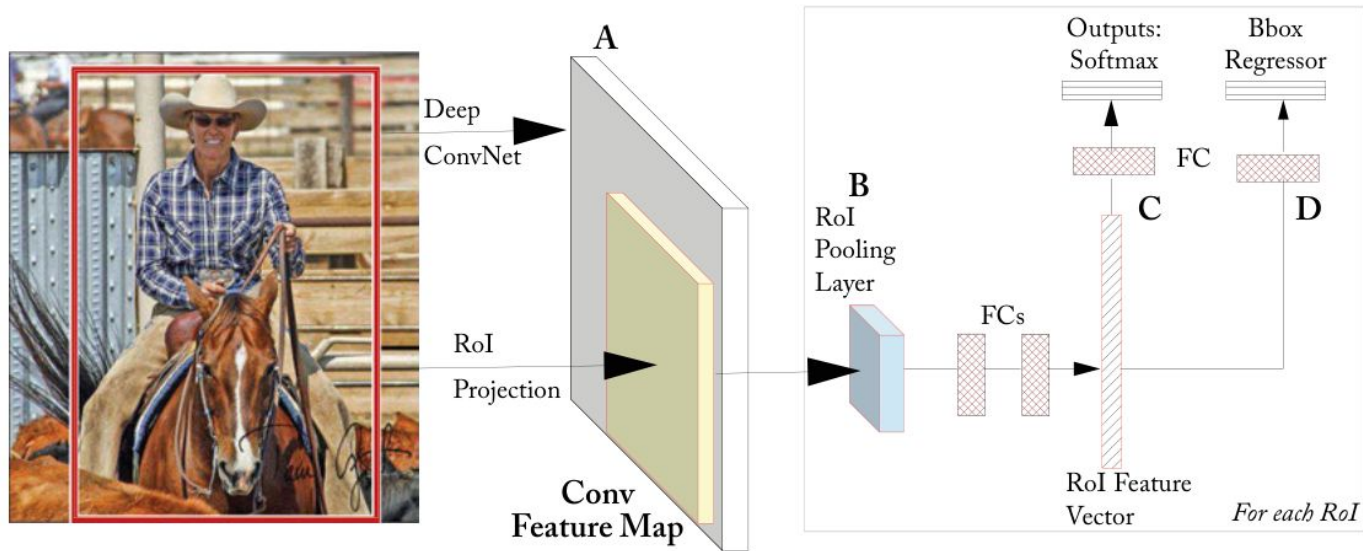


Figure 7.3: Fast R-CNN Architecture. The input to a fully convolutional network (e.g., AlexNet) is an image and RoIs (Module A). Each RoI is pooled into a fixed-size feature map (Module B) and then passed through the two fully-connected layers to form a RoI feature vector. Finally, the RoI feature vector is passed through two sibling layers, which output the soft-max probability over different classes (Module C) and bounding box positions (Module D).

Fast R-CNN es la evolución directa de la arquitectura R-CNN, desarrollada en 2015 por Ross Girshick con el objetivo de solucionar sus grandes problemas de lentitud y complejidad computacional, mejorando simultáneamente la velocidad y la precisión en la detección de objetos.

Su innovación principal radica en optimizar el flujo de trabajo mediante los siguientes cambios fundamentales respecto a su predecesor:

- **Procesamiento global de la imagen:** En lugar de recortar 2,000 regiones candidatas y pasarlas una por una por la red convolucional (lo cual era extremadamente lento), Fast R-CNN **pasa la imagen completa una sola vez** por la CNN para extraer un único mapa de características de manera global.
- **Capa RoI Pooling:** Las regiones propuestas (que aún se generan con el algoritmo de búsqueda selectiva) se identifican directamente sobre ese mapa de características global. Para poder procesar estas regiones que tienen diferentes tamaños, Fast R-CNN introduce una capa de agrupamiento de Regiones de Interés (*RoI pooling layer*), la cual extrae una ventana de tamaño fijo para cada región y la envía a las siguientes capas de la red.
- **Unificación del modelo (Dos cabezas):** Elimina el uso de algoritmos de clasificación externos (como los SVM) y unifica la tarea. La red neuronal finaliza ramificándose en dos salidas simultáneas: una capa *softmax* que predice la probabilidad de la clase del objeto, y un regresor que ajusta las coordenadas de la caja delimitadora (*bounding box*)

Faster R-CNN

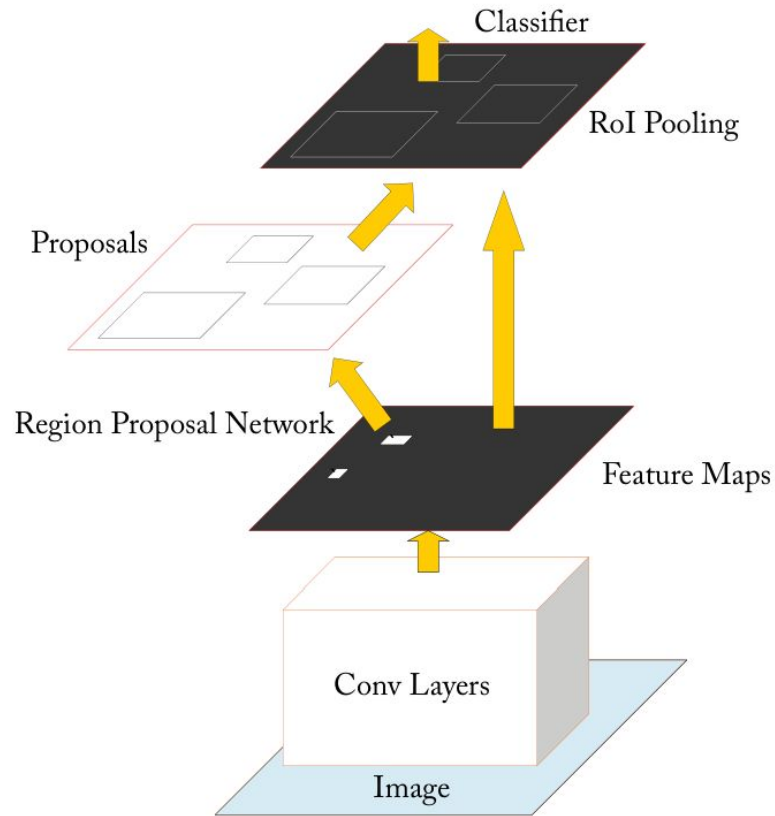


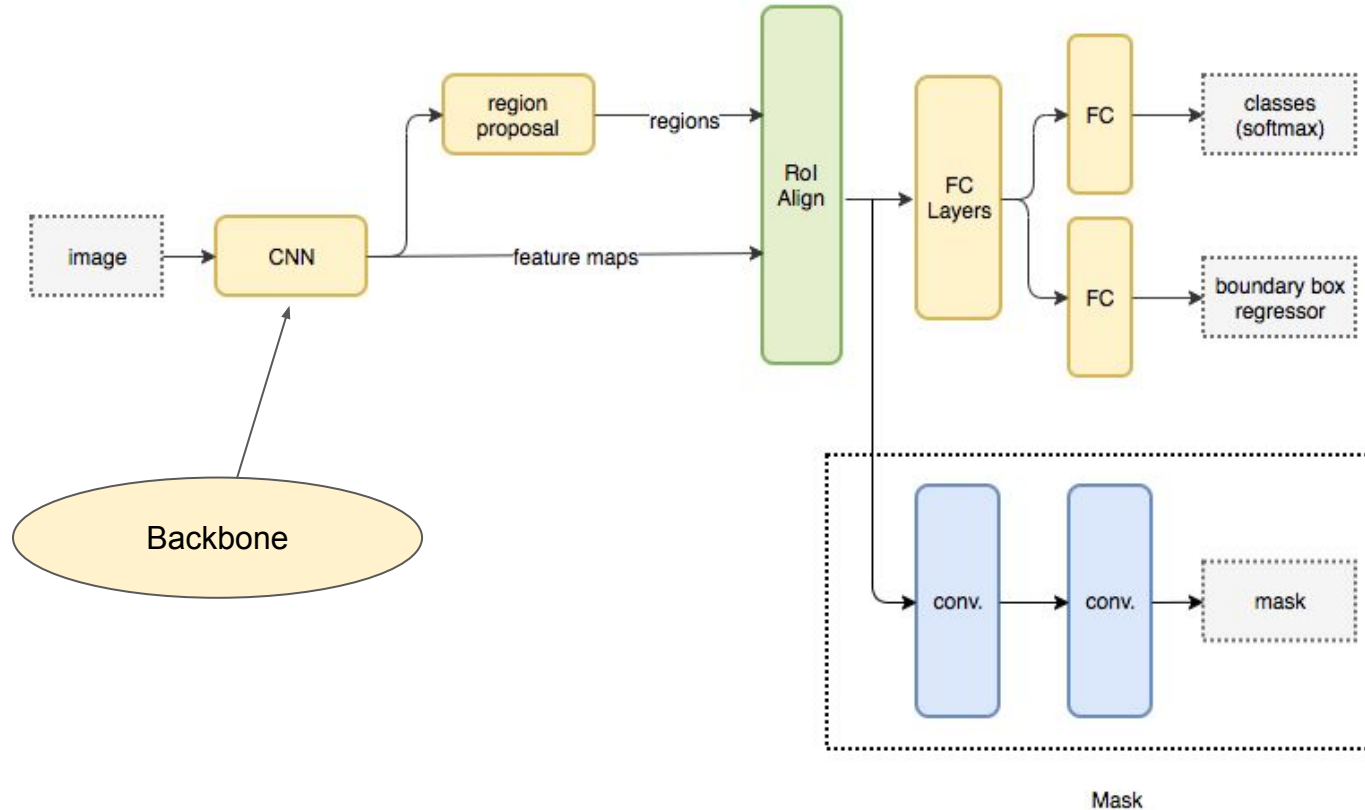
Figure 7.5: Faster R-CNN. Combining RPN with fast R-CNN object detector.

Faster R-CNN es la tercera iteración de la familia de detectores R-CNN. Su innovación fundamental es la introducción de una **Red de Propuesta de Regiones (RPN - Region Proposal Network)**.

Partes:

1. **Extracción de características (Red Base)**: Una red convolucional estándar procesa la imagen completa para extraer sus mapas de características visuales.
2. **Red de Propuesta de Regiones (RPN)**: Esta pequeña red convolucional opera directamente sobre los mapas de características generados en el paso anterior. Su función es predecir rápidamente regiones que tienen una alta probabilidad de contener un objeto (puntuación de objetividad) y delimitar sus coordenadas aproximadas mediante el uso de cajas de referencia predefinidas llamadas *anchors*.
3. **Capa RoI Pooling**: Al igual que en Fast R-CNN, esta capa toma las regiones de interés sugeridas por la RPN y las redimensiona a un tamaño fijo.
4. **Clasificación y Regresión**: Las regiones redimensionadas pasan por capas completamente conectadas que se dividen en dos salidas: un clasificador *softmax* que determina la clase del objeto detectado, y un regresor lineal que ajusta las coordenadas finales de la caja delimitadora.

Mask R-CNN



Mask R-CNN es un modelo avanzado que representa una evolución en la familia de arquitecturas orientadas a la propuesta de regiones (junto con R-CNN, Fast R-CNN y Faster R-CNN). Destaca principalmente por alcanzar resultados en el estado del arte (*state-of-the-art*) en tareas complejas de **detección de objetos y segmentación de instancias**.

1. Red Base (Backbone): Es una red neuronal convolucional profunda (generalmente arquitecturas como ResNet o ResNeXt, a menudo combinadas con una *Feature Pyramid Network* o FPN). Su función es procesar la imagen completa y extraer los mapas de características (patrones visuales) a diferentes escalas.

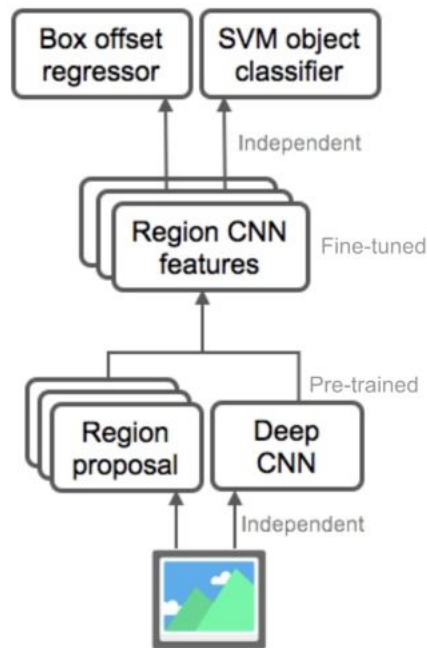
2. Red de Propuesta de Regiones (RPN - Region Proposal Network): Al igual que en Faster R-CNN, esta pequeña red se desliza sobre los mapas de características generados por la red base para identificar y proponer regiones candidatas (Rols) que tienen una alta probabilidad de contener un objeto.

3. Capa RoIAlign: Esta es la innovación estructural más crítica introducida por Mask R-CNN. Reemplaza a la antigua capa *RoI Pooling*. En versiones anteriores, al extraer las regiones candidatas se redondeaban las coordenadas de los píxeles, lo que causaba una pérdida de precisión espacial. **RoIAlign** utiliza interpolación matemática (bilineal) para extraer las características manteniendo la alineación espacial exacta, lo cual es un requisito indispensable para poder "dibujar" máscaras precisas a nivel de píxel.

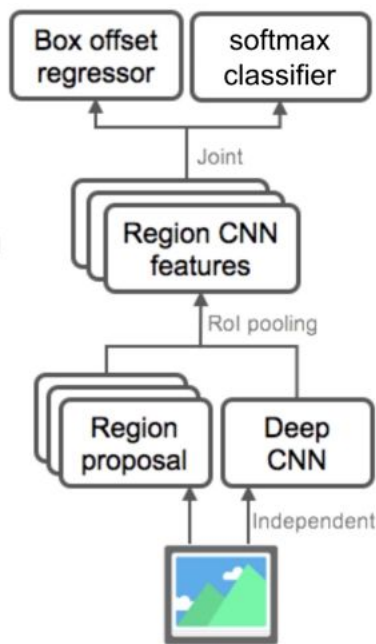
4. Las tres ramas de salida (Heads): Una vez que las características de cada región candidata han sido alineadas y extraídas mediante RoIAlign, la red se ramifica para realizar tres tareas en paralelo:

- **Rama de Clasificación:** Una capa de red neuronal tradicional que predice la probabilidad de que el objeto pertenezca a una clase específica (ej. persona, coche, perro).
- **Rama de Regresión de la Caja Delimitadora:** Un regresor que refina y ajusta las cuatro coordenadas ($x, y, ancho, alto$) del rectángulo que encierra al objeto.
- **Rama de Segmentación (Mask branch):** Es una pequeña Red Totalmente Convolutiva (FCN) añadida a la arquitectura. En lugar de devolver un simple valor numérico, devuelve una matriz espacial (una máscara binaria) que perfila la silueta exacta del objeto, indicando píxel por píxel qué partes de la caja delimitadora son el objeto y cuáles son el fondo.

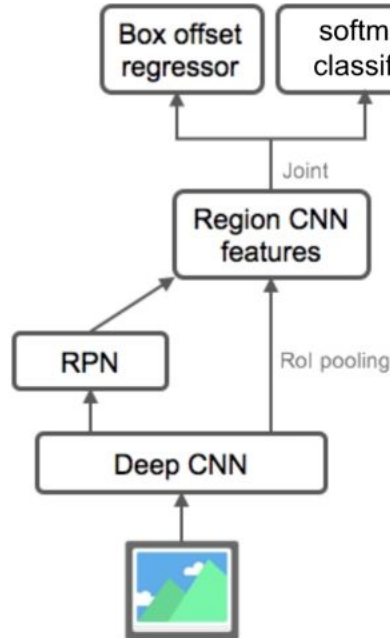
Resumiendo



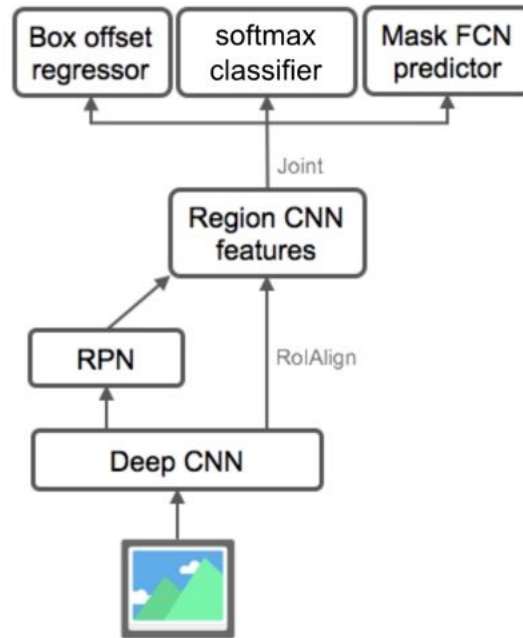
R-CNN



Fast R-CNN



Faster R-CNN



Mask R-CNN

2. Detectores de una etapa (Single-Stage) Estos modelos fueron creados para solucionar la lentitud de los detectores de dos etapas, saltándose la fase de proponer regiones explícitamente. Aquí destacan arquitecturas como **SSD (Single-Shot Detector)** y **YOLO (You Only Look Once)**. Estos algoritmos dividen la imagen de entrada en una cuadrícula y pasan la imagen completa una sola vez por la red convolucional (*single-shot*). En esa única pasada, la red predice las cajas delimitadoras y las probabilidades de clase para toda la imagen simultáneamente.

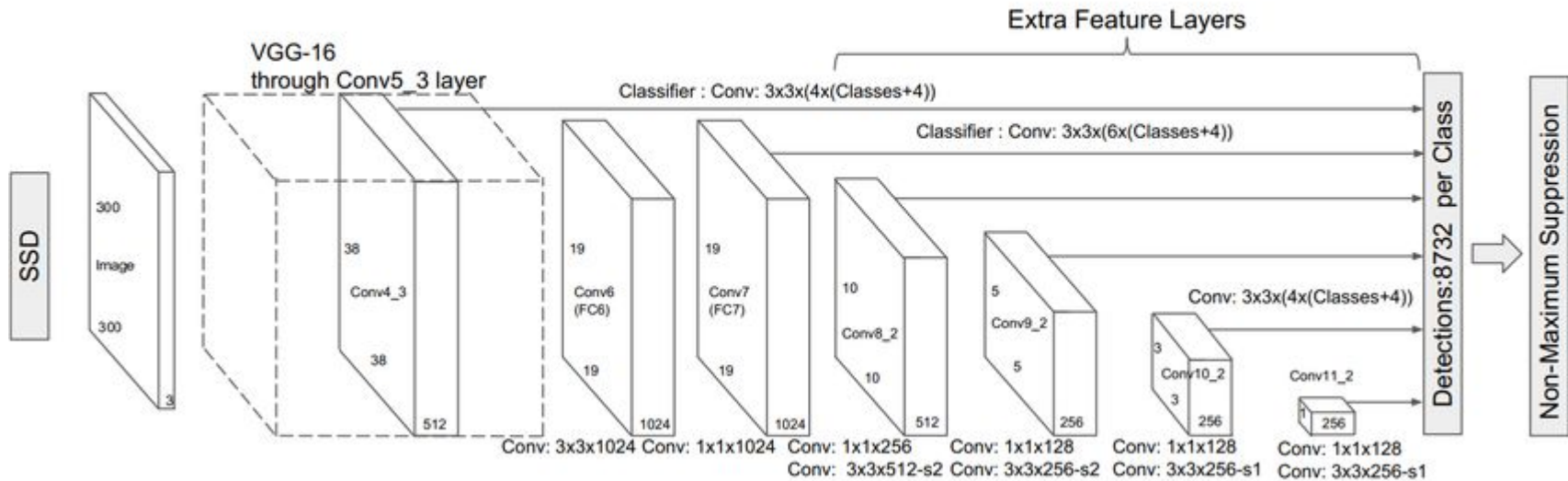
- **Ventaja:** Son excepcionalmente rápidos, alcanzando métricas de FPS que permiten aplicarlos en video en **tiempo real**, lo cual es clave para vehículos autónomos, seguridad o robótica.
- **Desventaja:** Históricamente, al analizar la imagen globalmente en un solo paso, pueden sacrificar un poco de precisión o tener ciertas dificultades detectando objetos muy pequeños, en comparación con la familia R-CNN.

El **SSD (*Single-Shot Detector*)**, presentado por Wei Liu y su equipo en 2016, es otro de los algoritmos clásicos y fundamentales para la detección de objetos. Al igual que YOLO, SSD es un **detector de una sola etapa (*single-stage detector*)**.

A diferencia de la familia de redes R-CNN, que utiliza múltiples etapas y es más lenta (primero propone regiones y luego las clasifica), SSD pasa la imagen una sola vez por la red neuronal para predecir, de manera simultánea, la ubicación de las cajas delimitadoras y a qué clase pertenece el objeto. Esto le permite ser extremadamente rápido, operando a unos 59 FPS (fotogramas por segundo), en contraste con los lentos 7 FPS que alcanzaba Faster R-CNN en su momento.

Partes:

1. Red Base (*Base Network*)
2. Capas de características a múltiples escalas (*Multi-scale feature layers*)
3. Predicción densa mediante *Priors* y NMS

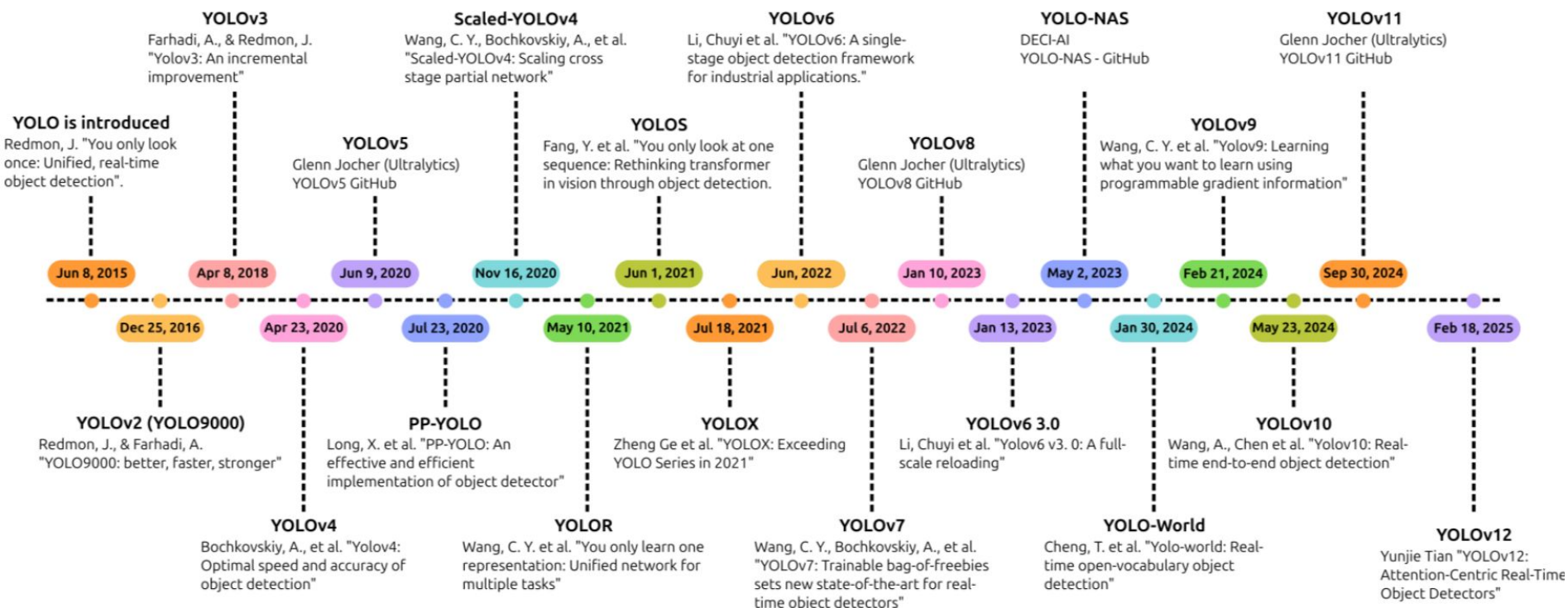


YOLO (You Only Look Once) es una familia de modelos diseñados específicamente para la **detección de objetos en tiempo real**. A diferencia de una clasificación simple, la detección de objetos no solo identifica qué hay en la imagen, sino que también localiza exactamente dónde está mediante el trazado de una caja delimitadora (*bounding box*) alrededor del objeto.

El éxito revolucionario de YOLO radica en que es un **detector de una sola etapa (*single-stage detector*)**. Mientras que los modelos clásicos de dos etapas (como R-CNN) primero proponen miles de posibles regiones y luego ejecutan un clasificador para cada una (lo cual es muy lento), YOLO reformuló la detección como un único problema de regresión. **"Mira la imagen solo una vez"**: los datos pasan a través de la red neuronal en una sola corrida (*forward pass*) para predecir todas las cajas y clases simultáneamente.



Evolución de YOLO



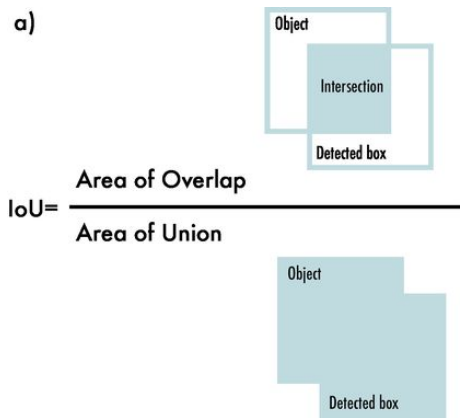
SSD vs YOLO

Característica	YOLO	SSD
Cómo funciona	Predice utilizando un pase basado en cuadrícula	Utiliza mapas de características de múltiples escalas
Velocidad	Generalmente más rápido	Un poco más lento que YOLO
Detección de objetos pequeños	Más débil en las primeras versiones	Mejor detección de objetos pequeños
Arquitectura	Red única unificada	Múltiples capas para predicción
Mejor caso de uso	Tareas en tiempo real	Aplicaciones que necesitan precisión de objetos pequeños
Compatibilidad del dispositivo	Necesita GPU moderada	Funciona bien incluso en dispositivos móviles

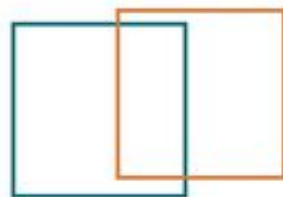
<https://www.geeksforgeeks.org/machine-learning/difference-between-yolo-and-ssd/>

1. Intersección sobre Unión (IoU - Intersection over Union) Es la métrica fundamental para evaluar la calidad de la localización espacial. El IoU mide la cantidad de solapamiento que existe entre la caja delimitadora predicha por el modelo y la caja real exacta (ground-truth).

- Se calcula dividiendo el área de intersección (solapamiento) de ambas cajas por el área total de unión de las mismas.
- Su valor varía de 0 (ningún solapamiento) a 1 (solapamiento perfecto del 100%).
- El IoU se utiliza para determinar si una predicción es válida (Verdadero Positivo) o no (Falso Positivo) fijando un umbral; por ejemplo, es un estándar requerir que el IoU sea mayor a 0.5 para considerar que el objeto fue detectado correctamente

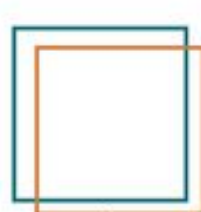


IoU= 0.35



Poor

IoU= 0.74



Good

IoU= 0.93

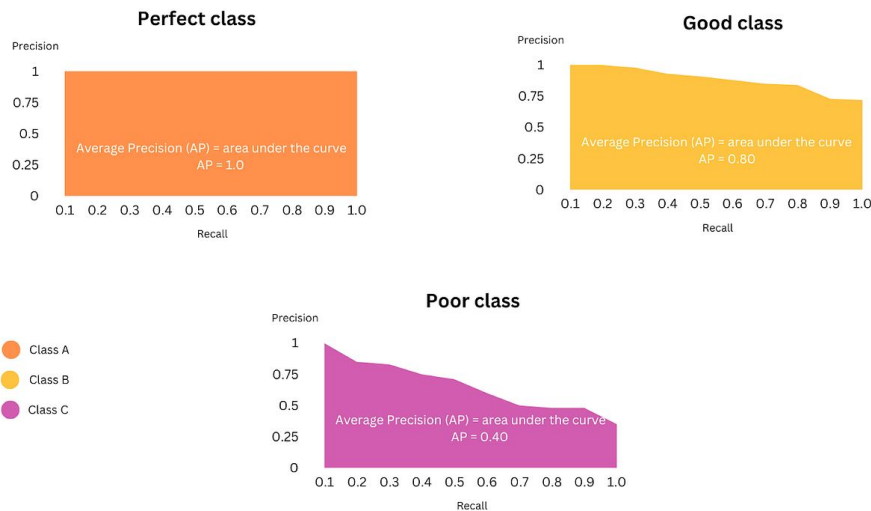


Excellent

2. Curva de Precisión-Exhaustividad (PR Curve - Precision-Recall Curve) Utilizando los verdaderos y falsos positivos definidos por el umbral de IoU, se calcula la precisión y la exhaustividad (recall) para cada clase.

- La curva PR grafica la precisión frente al recall variando el umbral de confianza del modelo.
- Un detector de objetos se considera excelente si su curva demuestra que la precisión se mantiene alta a medida que el recall también aumenta.

Precision - Recall curve



3. Precisión Media Promedio (mAP - Mean Average Precision) Es la métrica global más común y estandarizada para evaluar la precisión general de una red de detección de objetos. Para llegar a este número se realizan dos pasos:

- Primero, se calcula la **Average Precision (AP)** para cada clase individual, lo cual equivale matemáticamente a calcular el área bajo la curva PR de esa clase específica.
- Luego, se calcula el **mAP** promediando todos los valores de AP obtenidos en las distintas clases del problema.

Mean Average Precision Formula

$$\text{Mean Average Precision} = \frac{1}{n} \sum_{k=1}^{k=n} AP_k$$

n = the number of classes

AP_k = the average precision of class k

Métricas para detección de objetos

Object Detection Metrics Summary			Comments
IOU: matching score		Confidence: prediction confidence score	Fairly consistent definitions throughout the literature
IoU threshold: minimum IoU score for boxes to be considered "matched"		Confidence threshold: minimum confidence for prediction to be considered	
TP: correct prediction	FP: incorrect prediction	FN: missing prediction	
Precision: $TP/(TP + FP)$		Recall: $TP/(TP + FN)$	
P/R Curve: Curve showing Precision and Recall scores for different confidence and/or IoU threshold values			Some confusion as to whether this is plotted for varying IoUs with fixed confidence, or for varying confidences with fixed IoUs
AP: Average Precision. Area under the P/R Curve (i.e. "the average value of Precision for all recall values")		AR: Average Recall. Recall averaged over IoU thresholds in [0.5, 1.0]	Some confusion here. AR is not the average recall over all precision estimates, but is quite different.
mAP: AP averaged across all classes		mAR: AR averaged across all classes	Some confusion as to whether mAP and AP refer to the same thing or not.

Degrees of confusion about a given term:  Low Confusion  Moderate Confusion  High Confusion

<https://www.edge-ai-vision.com/2023/08/mean-average-precision-map-common-definition-myths-and-misconceptions/>

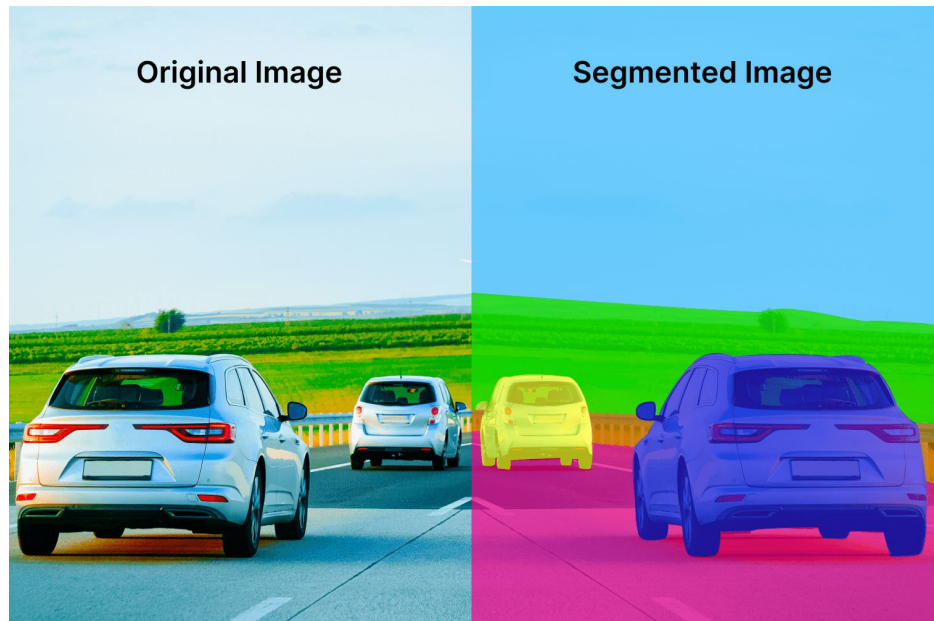
Segmentación de imágenes

U-Net

Segmentación semántica

La **segmentación semántica** es una tarea fundamental en la visión por computadora que consiste en **clasificar cada píxel individual de una imagen**, asignándole una etiqueta o categoría específica (por ejemplo, "gato", "perro", "fondo" o "calle").

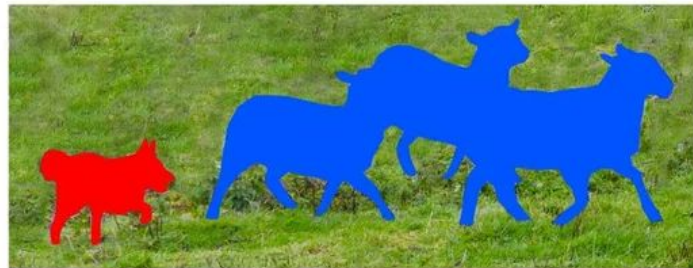
Mientras que la clasificación tradicional funciona como un embudo que toma todos los píxeles y te dice *qué* hay en la imagen general (ej. "hay un gato"), la segmentación semántica va un paso más allá y te dice *exactamente dónde* está ese objeto, delimitando sus bordes a nivel de píxel.



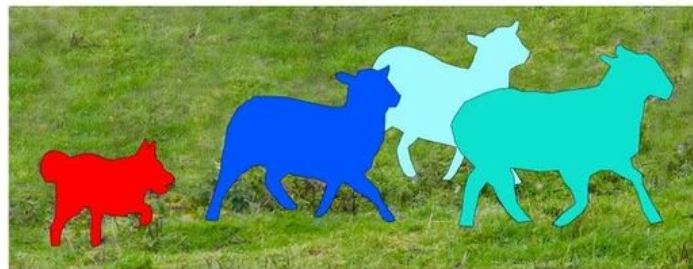
Segmentación de instancias

La **segmentación de instancias** busca no solo clasificar los píxeles de una imagen según su categoría, sino también **identificar, analizar y separar de forma individual cada objeto (o "instancia")** presente en la escena.

A diferencia de la segmentación semántica, que agrupa y etiqueta de manera uniforme todos los píxeles que pertenecen a una misma clase sin importar a qué objeto específico correspondan, la segmentación de instancias **logra distinguir entre objetos individuales de la misma clase.**



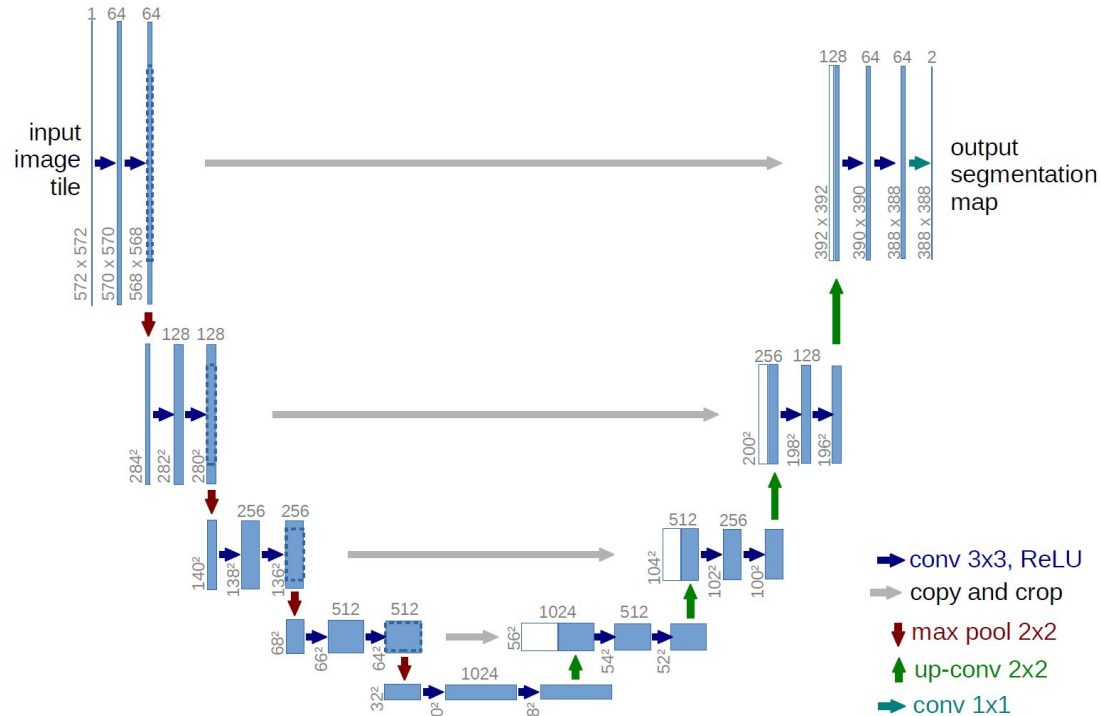
Semantic Segmentation



Instance Segmentation

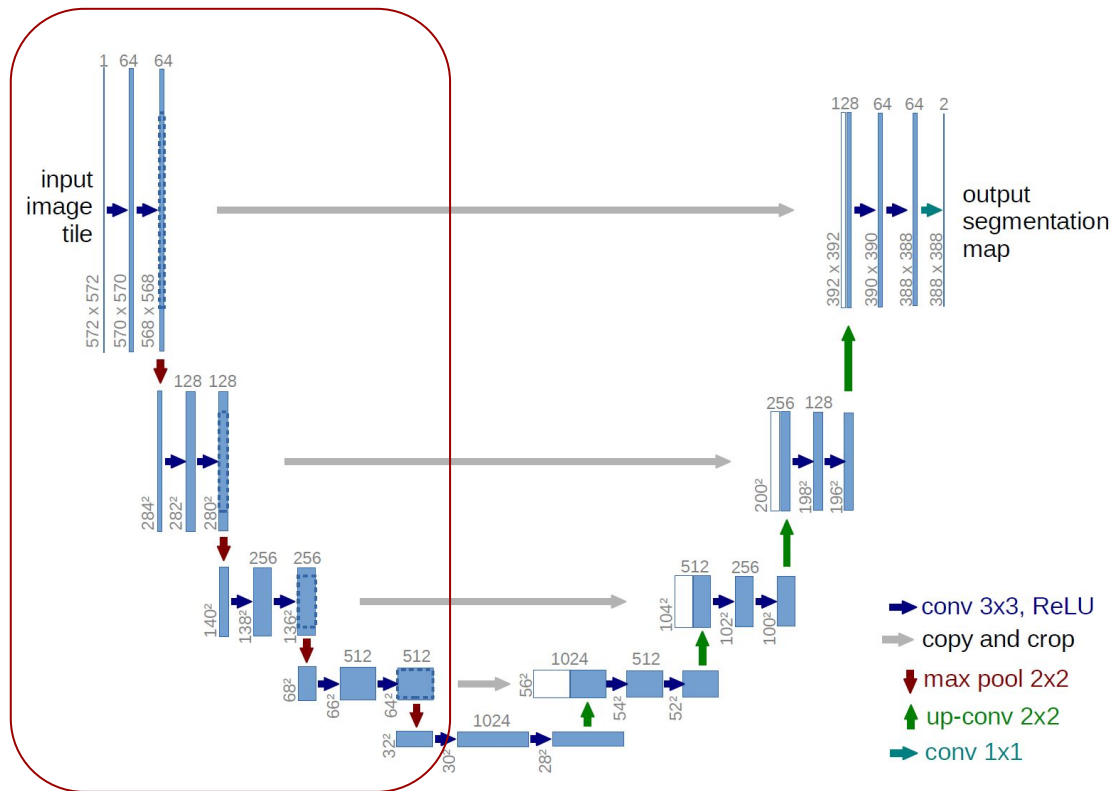
La arquitectura **U-Net** es un diseño de red neuronal convolucional que representó un avance fundamental para las tareas de **segmentación semántica de imágenes** (el proceso de clasificar individualmente cada píxel de una imagen para saber exactamente dónde se ubican los objetos). Fue desarrollada en 2015 por Olaf Ronneberger y su equipo, originalmente para segmentar imágenes biomédicas.

El nombre "U-Net" proviene directamente de la forma visual de su diagrama de arquitectura, el cual se asemeja a la letra "U". Esta estructura simétrica se divide en dos grandes mitades o rutas.



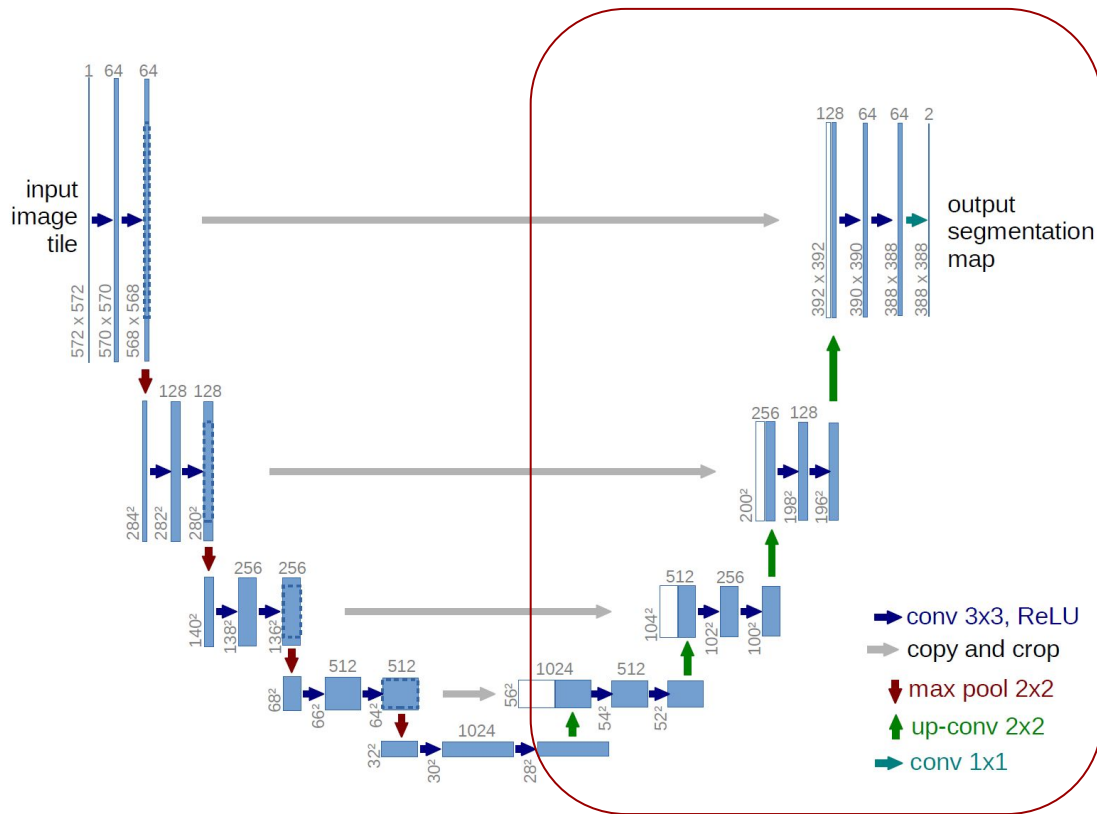
Partes de la U-Net

1. El camino de contracción (Codificador / Lado izquierdo): Funciona como una red convolucional tradicional (similar a VGG o AlexNet). Su objetivo es **extraer características y comprender el contexto** de la imagen. A medida que los datos fluyen hacia abajo por este lado, la red aplica convoluciones y capas de *pooling* (submuestreo) que reducen progresivamente la resolución espacial de la imagen (ancho y alto), pero aumentan la profundidad (el número de filtros o canales) para capturar información semántica cada vez más abstracta.



Partes de la U-Net

2. El camino de expansión (Decodificador / Lado derecho): Su objetivo es tomar esa información comprimida en el fondo de la "U" y **reconstruir la resolución espacial** para poder emitir un mapa de predicciones píxel por píxel. Para lograrlo, utiliza operaciones de "up-convolution" (convoluciones transpuestas o sobremuestreo) que van escalando la imagen hacia arriba hasta recuperar el tamaño original.



- **Conexiones saltadas (*Skip Connections*)** El gran problema de las redes de codificador-decodificador anteriores era que, al reducir tanto la imagen en el lado izquierdo, la red perdía la información espacial exacta (la ubicación precisa de los bordes de los objetos), lo que dificultaba la reconstrucción en el lado derecho.

U-Net solucionó esto introduciendo **conexiones transversales que cruzan directamente desde el lado izquierdo hacia el lado derecho** en cada nivel de resolución. Estas conexiones toman los mapas de características de alta resolución del codificador, los copian y los unen (concatenan) con las entradas de las capas correspondientes en el decodificador.

Gracias a esto, las capas de upsampling de la derecha no solo reciben la información semántica profunda de la capa anterior, sino que **también reciben los detalles espaciales finos directamente de las primeras capas de la red**. Esto es lo que le permite a U-Net dibujar máscaras y contornos precisos alrededor de los objetos segmentados.

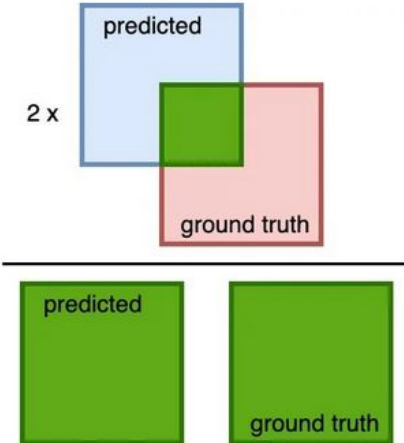
Otras características importantes:

- **Es una red totalmente convolucional (FCN):** Al no tener capas densas finales, preserva la resolución espacial y le permite aceptar imágenes de diversos tamaños como entrada (a menudo utilizando técnicas de relleno o *padding* para que la entrada coincida con la salida).
- **Eficiencia con pocos datos:** Fue diseñada específicamente para poder entrenarse de extremo a extremo utilizando conjuntos de datos muy pequeños, lo cual es vital en el ámbito médico donde las imágenes anotadas son escasas.

Funciones de pérdida específicas

- Dice Loss y Soft Dice
- Entropía cruzada por píxel (Per pixel cross entropy)

Dice coefficient = $\frac{2 \times \text{area of overlapped (green)}}{\text{total area (green)}} =$



$$\text{Dice Coefficient} = \frac{2 \times |X \cap Y|}{|X| + |Y|}$$

$$\text{Dice Loss} = 1 - \text{Dice Coefficient}$$

- X = Predicted segmentation
- Y = Ground truth segmentation

Pérdida Dice (Dice Loss) y Soft Dice Basada en el Coeficiente Sørensen-Dice, es la función de pérdida más común y recomendada para tareas de segmentación.

- **La ventaja principal:** Su mayor beneficio frente a otras funciones es que **maneja excepcionalmente bien el desbalance de clases en los datos de entrenamiento**. En muchos problemas típicos de U-Net (como detectar un pequeño nódulo en una tomografía computarizada completa), la inmensa mayoría de los píxeles pertenecen al "fondo" o tejido sano, y solo una fracción ínfima es el objeto positivo. Dice Loss evita que este desbalance arruine el entrenamiento.
- **Soft Dice:** Para que la red pueda aprender (ya que requiere funciones diferenciables para calcular gradientes), la pérdida Dice no utiliza las máscaras binarias finales (0 o 1). En su lugar, reemplaza el conteo de verdaderos positivos por la suma de las predicciones continuas (las probabilidades en coma flotante arrojadas por la red) en aquellos píxeles donde la verdad de campo (*ground-truth*) es 1. A este coeficiente Dice que utiliza predicciones continuas se lo conoce como **Soft Dice**.

Entropía Cruzada por Píxel (Per-pixel Cross-Entropy) Es la traslación de la función de pérdida clásica de clasificación de imágenes, pero aplicada de forma independiente a cada píxel individual de la máscara de salida para clasificarlo en una categoría (usualmente mediante una capa *softmax* al final del modelo).

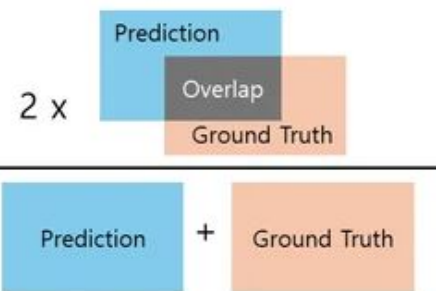
- **Limitaciones en segmentación:** Aunque es matemáticamente válida, sufre graves problemas ante el desbalance de clases. Si un tumor ocupa solo el 1% de la imagen, la red puede lograr una Entropía Cruzada muy baja simplemente prediciendo que el 100% de la imagen es "fondo", lo cual anula el propósito del modelo

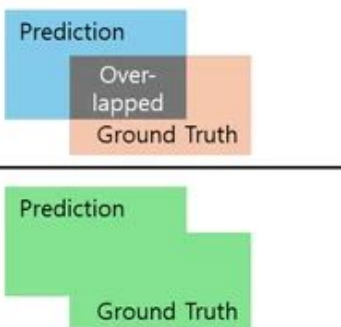
Imágenes Médicas y Diagnóstico (Su dominio original): Se utiliza masivamente para la delineación automática de estructuras anatómicas (como pulmones, hígados o materia blanca del cerebro) y la detección de estructuras patológicas. Es fundamental para cuantificar tumores cerebrales, segmentar lesiones isquémicas o encontrar pequeños nódulos de cáncer en tomografías computarizadas (CT) y resonancias magnéticas (MRI).

Vehículos Autónomos: Resulta crítica para la comprensión detallada de escenas (scene understanding). Se emplea para que el sistema del automóvil sepa a nivel de píxel exactamente dónde se ubican los peatones, otros vehículos, las señales de tránsito, las aceras y el espacio libre transitable de la calle

- **Edición de Video e Imágenes:** Funciones cotidianas, como el recorte de retratos o la capacidad de plataformas como Zoom y Google Meet para desenfocar o reemplazar el fondo durante una videollamada, utilizan segmentación de imágenes para distinguir tu rostro del entorno que te rodea con precisión de píxel.
- **Inspección Industrial y Manufactura:** En las líneas de ensamblaje, la segmentación es indispensable cuando la precisión espacial es crítica. Se usa para encontrar anomalías finas, delinear defectos en las superficies de los productos y poder medir exactamente el tamaño de dichas fallas.
- **Agricultura de Precisión y Biología:** Se aplica en tareas de fenotipado, como la segmentación de imágenes de raíces de plantas para analizar su crecimiento, longitud y estructura bajo diferentes condiciones de luz o suelo.
- **Robótica:** Dota a los sistemas robóticos de una comprensión espacial de grano fino, permitiéndoles interactuar con su entorno y manipular objetos complejos o con límites difíciles de delimitar.

Métricas para segmentación de imágenes

$$\text{Dice Similarity Coefficient (DSC)} = \frac{2 \times \text{Area of Overlap}}{\text{Total Area}} = \frac{2 \times \text{Overlap}}{\text{Prediction} + \text{Ground Truth}}$$


$$\text{Intersection over Union (IoU)} = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{\text{Over-lapped}}{\text{Prediction} \cup \text{Ground Truth}}$$


Métricas para segmentación de imágenes

Coeficiente Sørensen-Dice (o Coeficiente Dice) Es la métrica de evaluación (y frecuentemente también la función de pérdida) más común y estandarizada para tareas de segmentación.

- **Cómo se calcula:** Se basa en la proporción de píxeles segmentados correctamente frente a la suma total de píxeles predichos y reales. Se calcula multiplicando por dos el área de intersección (Verdaderos Positivos) y dividiéndolo por la suma del área predicha y el área real (*ground-truth*).

Intuición: En términos prácticos, el Coeficiente Dice funciona exactamente como un **F1-Score a nivel de píxel**.

- **La gran ventaja:** Su principal beneficio sobre métricas como la entropía cruzada es su **excelente manejo del desbalance de clases**. En problemas típicos de U-Net (como detectar un tumor en una tomografía o un defecto minúsculo en una pieza industrial), la inmensa mayoría de los píxeles de la imagen pertenecen al "fondo" y solo una fracción ínfima pertenece al objeto de interés. Dice evita que el modelo parezca tener un rendimiento excelente simplemente adivinando que todos los píxeles son "fondo".

Aplicaciones Espaciales

U-Net-Id, an Instance Segmentation model for building extraction from Satellite images - Case study in the Joanópolis city, Brazil

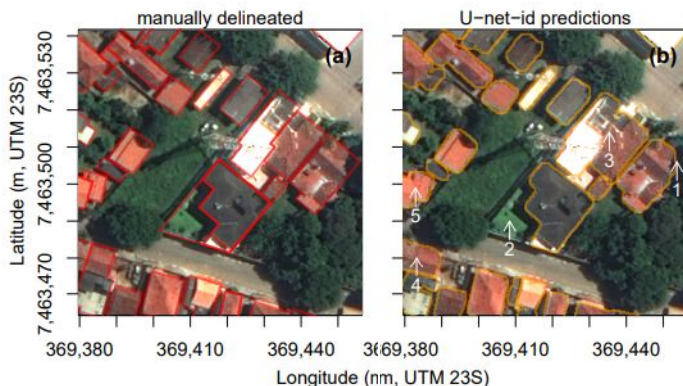
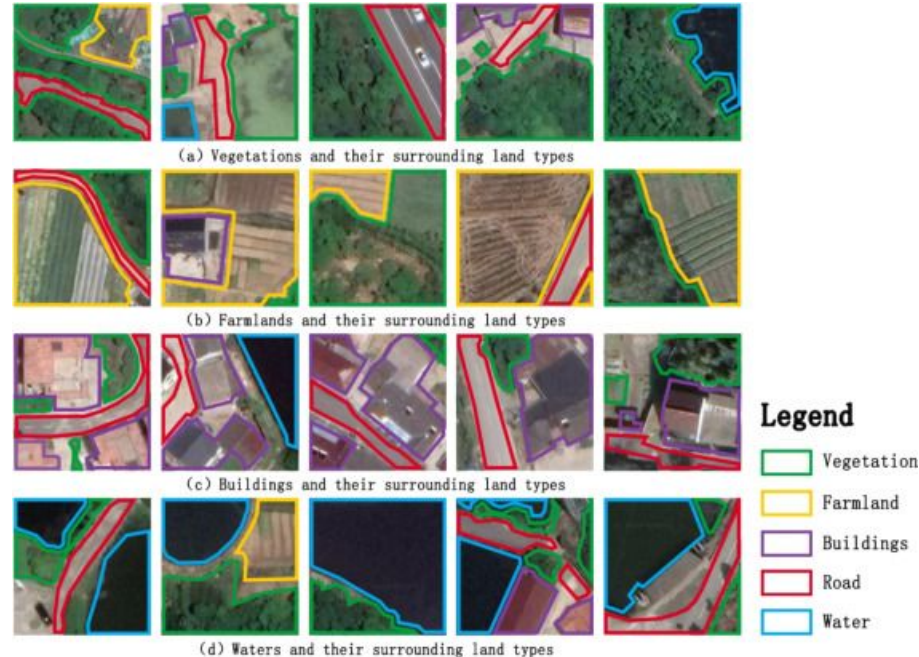
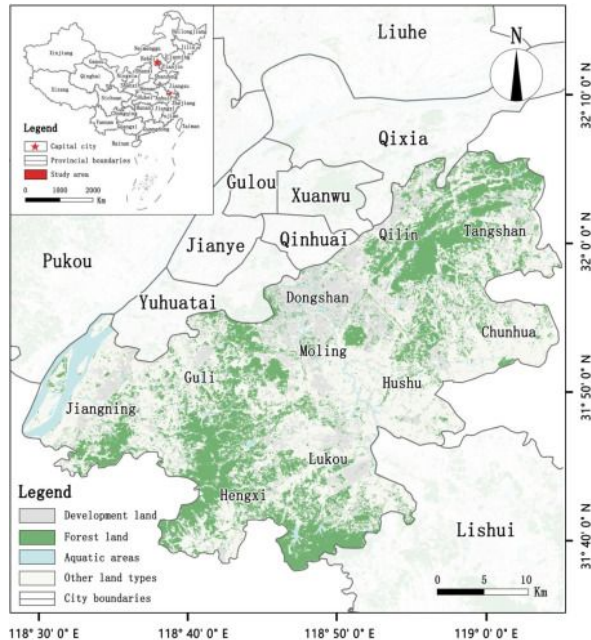


Table 2. Comparison of the building numbers and areas between the Joanópolis building dataset (observed) and the buildings instances predicted by the U-net-id model (predicted).

	Observed	Predicted
<i>Worldview tile extent</i>		
Number of buildings	unknown ^a but > 7563	7477
<i>Joanópolis main extent</i>		
Number of buildings	7380	6106
Building total area (m ²)	639,285	618,077
Mean building size (m ²)	86.6	101.2
Median building size (m ²)	68.6	78.4

^a In the reference dataset, not all buildings in the extent covered by the Worldview tile have been manually delineated.

Land-Unet: a deep learning network for precise segmentation and identification of non-structured land use types in rural areas for green urban space analysis



Chronological review and performance analysis of YOLO-based deep learning frameworks in complex geospatial environments

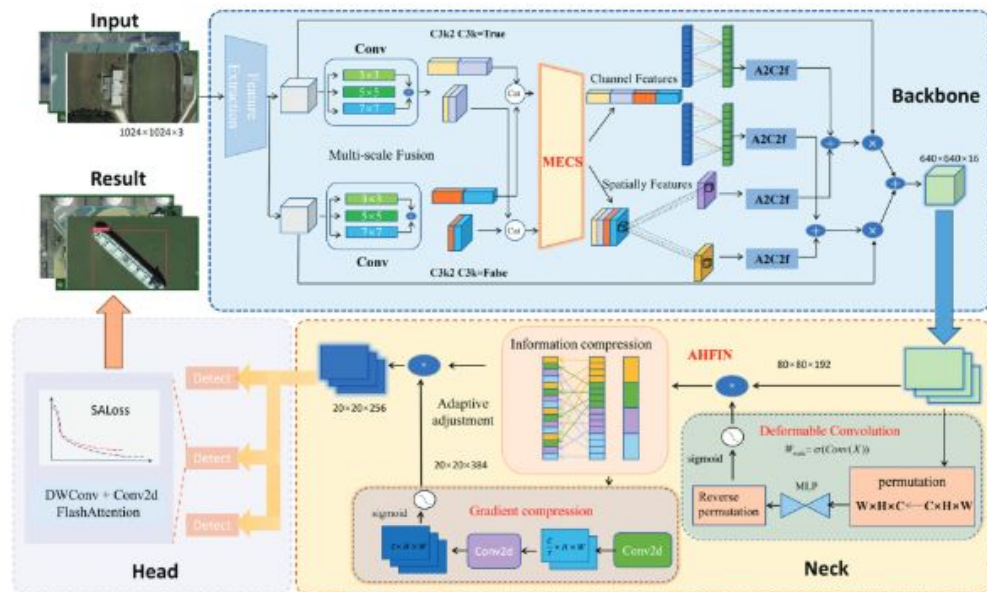


Figure 2. Overview pipeline of YOLOv12 for geospatial object detection (input image → YOLO model → GIS output).





<https://www.jeremyjordan.me/semantic-segmentation/>

https://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf

<https://github.com/uav4geo/GeoDeep>

https://wvview.org/dl/pytorch_examples/quarto/T14_Train_UNet.html

Agregar

Benchmarks para segmentacion de imagenes

Ampliar la parte de segmentacion de imagenes

Colab: segmentación de edificios